→ Multiple model training, pipeline and Column Transformer.

```
import panda as pd
import numpy as np
import Matplotlib.pyplot as plt
import seaborn as sns

import warnings
warning.filterwarnings("ignore")

df = sns.load_dataset('tips')
df.head()
```

→ ingest th data.
→ EDA
→ X, y
→ train_test_split.

Automate Now In this. { missing value treatment, outlier treatment, Scaling }

→ Model traing.
→ Evaluation.

predicty what is the time ?. If any one. is visiting -- is it Lunch or dinner ?. time is the target variable.

```
df.time.unique()

df.info()

df.isnull().sum()
```

Automate →
→ missing value.
→ outlier treatment
→ Scaling

No Null, Still we will see how to null value imputation

→ Since time is nominal variable, then we will use label encoder

```
from Sklearn.preprocessing import LabelEncoder
encoder = LabelEncoder()
df['time'] = encoder.fit_transform(df['time'])

df['time'].unique()   # dinner = 1, lunch = 0
```

X = df.drop('time', axis=1)

y = df['time']

→ from sklearn.model_selection import train_test_split

→ X_test, X_train, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=1)

we Doing 3 things ① handling missing value.
② Data encoding
③ feature scaling

missing value

① → from Sklearn.impute import SimpleImputer

② for encoding

→ from Sklearn.preprocessing import OneHot encoder

③ Scaling.

→ from sklearn.preprocessing import StandardScaler.

Pipeline ⇒ Using Sklearn pipeline help to do all the 3 process. missing value, Data encoding and scaling be done. on coloum one by one.

→ from sklearn.pipeline import pipeline (# a sequence of data transform)

From Sklearn.compose import ColumnTransform

Group all the pipeline (3) steps in all the coloumn of table. Automatically

df.sample (1)

Saparate Numerical and Catagorical Column.

```
cat_cols = ["sex", "smoker", "day"]
num_cols = ["total_bill", "tip", "size"]
```

feature engineering automation using pipeline and Column transformer.

```
→ num_pipeline = Pipeline(step = [('imputation', SimpleImputer(strategy='median')),
                                  ('scaling', StandardScaler())])
                                                  ↓ object
                                  object.

→ cat_pipeline = Pipeline(steps = [('imputation', SimpleImputer(strategy='most_frequent')),
                                   ('Encoding', OneHotEncoder())])

preprocessor = ColumnTransformer([("num_pipeline", num_pipelinee, num_cols),
                                  ("cat_pipeline", cat_pipeline, cat_cols)])

X_train = Preprocessor.fit_transform(X_train)
X_test = preprocessor.transform(X_test)
```

New X train, X test value we Got

Now Data is Roade build Multiple Model.

```
from sklearn. tree import Decision Tree Classifier.
from sklearn. svm import SVC
from sklearn. linear_model import Logistic Regression

model = { "support Vector classifier": SVC(),
          "DT Classifier": Decision Tree Classifier(),
          "Log reg": Logistic Regression }

from sklearn. metrics import accuracy_score.
```

```
-> def model_train_eval(X_train, X_test, y_train, y_test, model
      evalution = { }
      for i in range(len(models)):
        model = list(models. values())[i]
        model. fit(X_train, y_train)
        y_pred = model. predict(X_test)
        model_score = accuracy_score(y_test, y_pred)
        evalution [list(models. Keys())[i]] = model_score.
      return evalution

model_train_eval(X_train, X_test, y_train, y_test, Models)
```